DevOps Shack

Microservices CI/CD Pipeline

GitHub Actions + GHCR + AWS EKS

Line-by-Line Deep Dive

A complete walkthrough of every stage, job, and step in the pipeline

1. Pipeline Overview

This GitHub Actions workflow automates the full CI/CD lifecycle for a Google microservices demo application — from secret scanning all the way to deploying to Amazon EKS. It is named Microservices GHCR CICD.

1.1 Trigger Configuration

```
on:
  push:
    branches: [ main ]
    paths-ignore:
      - 'release/'
```

The pipeline fires on every push to the main branch. The paths-ignore directive tells GitHub Actions to skip the pipeline entirely if ONLY files inside the release/ folder changed — preventing infinite loops caused by the pipeline's own manifest commits later.

> **Why paths-ignore on release/?**
> The update-manifest job commits updated Kubernetes image tags back to the repo. Without this exclusion, that commit would re-trigger the pipeline, causing an endless loop.

1.2 Permissions

```
permissions:
  contents: write
  packages: write
  security-events: write
```

contents: write	Allows the workflow to commit manifest updates back to the repository.
packages: write	Allows pushing Docker images to GitHub Container Registry (GHCR).
security-events: write	Allows uploading SARIF security scan results to GitHub Security tab.

1.3 Global Environment Variables

```
env:
  REGISTRY: ghcr.io
  USERNAME: jaiswaladi246
```

These env vars are available to every job. REGISTRY points to GitHub Container Registry and USERNAME is the GHCR namespace/account where images are pushed.

1.4 Job Execution Flow

The pipeline runs 6 jobs in a strict dependency chain:

#	Job	Purpose
1	`gitleaks`	Scan for hardcoded secrets and credentials in the codebase
2	`checkov`	Static analysis of IaC files (Dockerfiles, Kubernetes YAML, Terraform)
3	`detect-changes`	Use path filtering to identify which microservices changed

4	`service-pipeline`	For each changed service: FS scan, build Docker image, image scan, push to GHCR
5	`update-manifest`	Update Kubernetes manifest with new image tags and commit back to repo
6	`deploy-to-k8s`	Apply updated manifest to EKS cluster and verify rollout

2. Job: gitleaks — Secret Scanning

```
gitleaks:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Run Gitleaks
      run: |
        docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:latest detect \
          --source=/repo \
          --report-format json \
          --report-path /repo/gitleaks-report.json \
          --exit-code 0
```

2.1 What Gitleaks Does

Gitleaks is an open-source SAST tool that scans Git repositories for hardcoded secrets — API keys, passwords, tokens, private keys, AWS credentials, and hundreds of other secret patterns defined by built-in regex rules.

2.2 Line-by-Line Breakdown

runs-on: ubuntu-latest	Runs on a fresh GitHub-hosted Linux runner.
actions/checkout@v4	Clones the repo into the runner's workspace at $PWD.
docker run --rm	Runs the Gitleaks container and removes it after completion (--rm).
-v "$PWD:/repo"	Mounts the current workspace directory into the container at /repo.
zricethezav/gitleaks:latest	Official Gitleaks Docker image, always pulls the latest version.
detect	Gitleaks command: scan mode (as opposed to 'protect' for pre-commit hooks).
--source=/repo	Tells Gitleaks to scan the mounted directory /repo.
--report-format json	Outputs the findings as JSON for easy parsing/integration.
--report-path /repo/gitleaks-report.json	Saves the report back to the workspace (mounted volume) so it persists.
--exit-code 0	CRITICAL: Forces Gitleaks to exit 0 (success) even if secrets are found. This makes the scan non-blocking — the pipeline continues. Change to 1 to enforce a hard block on secrets.

Design Decision: --exit-code 0

Using exit-code 0 means this is an auditing scan, not a gate. It logs findings without stopping the pipeline. For production security enforcement, switch to --exit-code 1.

3. Job: checkov — IaC Static Analysis

```
checkov:
  runs-on: ubuntu-latest
  needs: gitleaks
  steps:
    - uses: actions/checkout@v4

    - name: Run Checkov
      run: |
        docker run --rm -v "$PWD:/tf" bridgecrew/checkov \
          -d /tf --soft-fail
```

3.1 What Checkov Does

Checkov is a static analysis tool for Infrastructure-as-Code. It checks Dockerfiles, Kubernetes YAML, Terraform, Helm charts, and more against hundreds of security and compliance policies — looking for misconfigurations like privilege escalation, missing resource limits, insecure network policies, and more.

3.2 Line-by-Line Breakdown

needs: gitleaks	This job only starts after gitleaks completes successfully. Enforces sequential order.
-v "$PWD:/tf"	Mounts the entire workspace into /tf inside the Checkov container.
bridgecrew/checkov	Official Checkov Docker image published by Bridgecrew (now Prisma Cloud).
-d /tf	Tells Checkov to recursively scan everything in /tf (the full repo).
--soft-fail	Like exit-code 0 in Gitleaks — reports findings but always exits 0, so the pipeline continues regardless of failures.

Checkov will scan every file it recognizes: Dockerfiles, .yaml manifests, Terraform .tf files, and more. It prints a table of passed, failed, and skipped checks.

> **What Checkov Catches**
> Common findings include: containers running as root, missing CPU/memory limits on pods, privileged containers, hostPath volume mounts, missing network policies, images using :latest tag without digest pinning, and insecure capabilities.

4. Job: detect-changes — Smart Matrix Builder

This is the most architecturally important job. It figures out which of the 12 microservices actually changed in this push, then builds a dynamic matrix so only those services go through the expensive build-scan-push pipeline.

4.1 Outputs Declaration

```
detect-changes:
  runs-on: ubuntu-latest
  needs: checkov
  outputs:
    matrix: ${{ steps.set-matrix.outputs.matrix }}
    has_changes: ${{ steps.set-matrix.outputs.has_changes }}
```

outputs:	Declares job-level outputs that downstream jobs can read via needs.detect-changes.outputs.
matrix	A JSON string like {"service":["frontend","cartservice"]} that drives the matrix strategy in service-pipeline.
has_changes	A boolean string ('true'/'false') that gates the update-manifest and deploy jobs.

4.2 First Run Detection

```
    - name: Detect first run
      id: first-run
      run: |
        if git rev-parse HEAD~1 >/dev/null 2>&1; then
          echo "first_run=false" >> "$GITHUB_OUTPUT"
        else
          echo "first_run=true" >> "$GITHUB_OUTPUT"
        fi
```

On the very first push to a repo, there is no parent commit (HEAD~1 does not exist). git rev-parse HEAD~1 fails, which means paths-filter cannot do a diff — there's nothing to compare against. The first-run detection catches this edge case.

git rev-parse HEAD~1	Tries to resolve the parent commit hash. Outputs to /dev/null to suppress output.
>/dev/null 2>&1	Redirects both stdout and stderr to null — we only care about the exit code.
echo ... >> $GITHUB_OUTPUT	The modern GitHub Actions way to set step outputs. Replaces the deprecated set-output command.

4.3 Path Filtering with dorny/paths-filter

```
    - id: filter
      uses: dorny/paths-filter@v3
      with:
        filters: |
          adservice:
            - 'src/adservice/**'
            - 'protos/**'
          cartservice:
```

```
            - 'src/cartservice/**'
            - 'protos/**'
          ... (repeated for all 12 services)
```

dorny/paths-filter compares changed files in the push against the glob patterns defined under filters. For each named filter (adservice, cartservice, etc.), it sets an output of 'true' or 'false'.

Notice that protos/** is listed for most services. The protobuf definitions are shared across services — if protos change, all dependent services need to be rebuilt.

> **Why protos/** appears multiple times**
> The microservices communicate via gRPC using shared .proto files. A change in a proto definition could break any service that uses it, so all those services must be rebuilt and redeployed together.

4.4 Building the Dynamic Matrix

```
    - name: Build dynamic matrix
      id: set-matrix
      run: |
        SERVICES=()

        [[ "${{ steps.filter.outputs.adservice }}" == "true" ]] &&
SERVICES+='"adservice"'
        [[ "${{ steps.filter.outputs.cartservice }}" == "true" ]] &&
SERVICES+='"cartservice"'
        ... (repeated for all services)

        if [ ${#SERVICES[@]} -eq 0 ]; then
          if [ "${{ steps.first-run.outputs.first_run }}" = "true" ]; then
            SERVICES=(all 12 services)
            HAS_CHANGES=true
          else
            SERVICES='"__none__"'
            HAS_CHANGES=false
          fi
        else
          HAS_CHANGES=true
        fi

        JSON="{\"service\":[$(IFS=,; echo "${SERVICES[*]}")]}"
        echo "matrix=$JSON" >> "$GITHUB_OUTPUT"
        echo "has_changes=$HAS_CHANGES" >> "$GITHUB_OUTPUT"
```

Logic Flow

SERVICES=()	Initialize an empty bash array to collect changed service names.
[[... == 'true']] && SERVICES+=	Conditional append: only add a service to SERVICES if its filter output is 'true'.
${#SERVICES[@]} -eq 0	Check if the array is empty — no services changed.
first_run=true branch	On first run with no parent commit, add ALL 12 services to ensure everything gets built.
SERVICES='"__none__"'	Sentinel value: when nothing changed, inject a dummy entry so the matrix job still runs but skips all steps.
HAS_CHANGES=false	Signal to downstream jobs (update-manifest, deploy) to not run.

IFS=,; echo "${SERVICES[*]}"	Join array elements with commas to produce valid JSON array items.
JSON={"service":[...]}	Construct the matrix JSON: {"service":["adservice","frontend",...]}

The __none__ Sentinel Pattern

GitHub Actions requires a matrix to have at least one entry. When nothing changed, injecting __none__ satisfies this requirement. Every step in service-pipeline then has an 'if: matrix.service != __none__' guard to skip actual work.

5. Job: service-pipeline — Build, Scan & Push

This is the core CI job. It runs in parallel for every service detected by detect-changes, executing 6 steps per service: skip guard, checkout, GHCR login, filesystem scan, Docker build, image scan, push, and cleanup.

5.1 Matrix Strategy

```
service-pipeline:
  runs-on: ubuntu-latest
  needs: detect-changes
  strategy:
    fail-fast: false
    matrix: ${{ fromJson(needs.detect-changes.outputs.matrix) }}
```

fail-fast: false	If one service's build fails, keep building the other services. Without this, a single failure would abort all parallel builds.
fromJson(...)	Deserializes the matrix JSON string output from detect-changes back into a real GitHub Actions matrix object.
needs.detect-changes.outputs.matrix	Reads the job-level output from detect-changes. Contains a list like ["adservice","frontend"].

5.2 Step-by-Step Breakdown

Step 1: Skip Guard

```
    - name: Skip when no service changed
      if: matrix.service == '__none__'
      run: echo "No service changes. Skipping build."
```

When the __none__ sentinel is in the matrix, this step prints a message and all subsequent steps are skipped by their own if guards.

Step 2: Checkout

```
    - uses: actions/checkout@v4
      if: matrix.service != '__none__'
```

Clones the repository. Every step from here has the same if guard to completely skip work when there are no changes.

Step 3: GHCR Login

```
    - name: Login GHCR
      if: matrix.service != '__none__'
      run: echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${{
github.actor }} --password-stdin
```

secrets.GITHUB_TOKEN	Auto-generated token that GitHub provides to every workflow run. No manual secret setup needed.
github.actor	The GitHub username that triggered the workflow — used as the registry username.
--password-stdin	Security best practice: reads the password from stdin instead of passing it as a flag (which would expose it in process listings).

| (pipe)	Pipes the echo output directly to docker login's stdin, avoiding the token ever appearing in shell history.

Step 4: Install Trivy

```
    - name: Install Trivy
      if: matrix.service != '__none__'
      uses: aquasecurity/setup-trivy@v0.2.6
      with:
        version: latest
        cache: true
```

Uses the official Aqua Security Trivy setup action to install Trivy on the runner. cache: true saves the Trivy vulnerability database between runs, significantly speeding up scans.

Step 5: Trivy Filesystem Scan

```
    - name: Trivy FS Scan
      if: matrix.service != '__none__'
      run: |
        if [ "${{ matrix.service }}" = "cartservice" ]; then
          trivy fs ./src/cartservice/src --severity HIGH,CRITICAL --exit-code 0
        else
          trivy fs ./src/${{ matrix.service }} --severity HIGH,CRITICAL --exit-code
0
        fi
```

Trivy fs scans the source code and dependency files (package.json, go.mod, requirements.txt, etc.) BEFORE building the Docker image. This catches known CVEs in dependencies early.

cartservice special case	cartservice has a non-standard layout: its source is inside src/cartservice/src/ (an extra src/ subdirectory), not at src/cartservice/. The conditional handles this.
--severity HIGH,CRITICAL	Only report High and Critical severity CVEs. Ignores Low and Medium to reduce noise.
--exit-code 0	Non-blocking scan — reports vulnerabilities but does not fail the pipeline. Change to 1 to enforce.

Step 6: Build Docker Image

```
    - name: Build Docker image
      if: matrix.service != '__none__'
      run: |
        if [ "${{ matrix.service }}" = "cartservice" ]; then
          docker build \
            -t $REGISTRY/$USERNAME/${{ matrix.service }}:${{ github.sha }} \
            ./src/cartservice/src
        else
          docker build \
            -t $REGISTRY/$USERNAME/${{ matrix.service }}:${{ github.sha }} \
            ./src/${{ matrix.service }}
        fi
```

$REGISTRY/$USERNAME/${{ matrix.service }}	Full GHCR image name: ghcr.io/jaiswaladi246/adservice (for example).

${{ github.sha }}	The full 40-character Git commit SHA. Using the commit SHA as the tag makes every build uniquely traceable to its exact source code state. Never use :latest in production pipelines.
cartservice src path	Same special case as Trivy: build context points to src/cartservice/src/ instead of src/cartservice/.

Step 7: Trivy Image Scan

```
    - name: Trivy Image Scan
      if: matrix.service != '__none__'
      run: |
        trivy image ghcr.io/${{ env.USERNAME }}/${{ matrix.service }}:${{ github.sha
}} \
          --severity HIGH,CRITICAL --exit-code 0
```

Scans the built Docker image — not just source files. This catches vulnerabilities in base images, OS packages, and system libraries that a filesystem scan would miss.

> **Two-Layer Scanning Strategy**
> FS scan (Step 5) catches application dependency CVEs before building. Image scan (Step 7) catches OS-level and base image CVEs after building. Together they provide comprehensive coverage.

Step 8: Push Image

```
    - name: Push Image
      if: matrix.service != '__none__'
      run: |
        docker push ghcr.io/${{ env.USERNAME }}/${{ matrix.service }}:${{ github.sha
}}
```

Pushes the built and scanned image to GHCR. The image is now available for the Kubernetes cluster to pull during deployment.

Step 9: Cleanup Docker

```
    - name: Cleanup Docker
      if: matrix.service != '__none__'
      run: |
        docker image prune -af
```

Removes all unused Docker images from the runner. GitHub-hosted runners have limited disk space. Without cleanup, multiple parallel service builds could exhaust disk and cause failures.

6. Job: update-manifest — GitOps Manifest Update

This job implements the GitOps principle: the desired state of the cluster (image tags) is stored in Git. It updates the Kubernetes manifest file with the new image SHAs and commits the change back to the repository.

```
update-manifest:
  runs-on: ubuntu-latest
  needs:
    - detect-changes
    - service-pipeline
  if: needs.detect-changes.outputs.has_changes == 'true'
```

needs: [detect-changes, service-pipeline]	Waits for BOTH jobs to complete. Images must be pushed before manifests are updated.
if: has_changes == 'true'	Skips entirely if no services changed. No point updating manifests if nothing was rebuilt.

6.1 Checkout with Full History

```
      - uses: actions/checkout@v4
        with:
          ref: main
          fetch-depth: 0
```

ref: main	Explicitly checks out the main branch (important: later steps do git push to main).
fetch-depth: 0	Fetches the complete git history. Needed so the subsequent git pull --rebase doesn't fail due to incomplete history.

6.2 Sed-based Image Tag Update

```
      - name: Update manifest images
        run: |
          SHA=${GITHUB_SHA}
          MATRIX='${{ needs.detect-changes.outputs.matrix }}'

          for svc in $(echo "$MATRIX" | jq -r '.service[]'); do
            if [ "$svc" = "__none__" ]; then
              continue
            fi
            sed -i "s|ghcr.io/${{ env.USERNAME }}/$svc:.*|ghcr.io/${{ env.USERNAME
}}/$svc:${SHA}|g" release/kubernetes-manifests.yaml
          done
```

MATRIX='...'	Reads the JSON matrix string from the detect-changes job output.
jq -r '.service[]'	Uses jq to parse the JSON and extract each service name as a plain string on its own line.
if ["$svc" = "__none__"]	Skips the sentinel value — never try to update a manifest for __none__.
sed -i	In-place edit: modifies the file directly without creating a temp file.

s|old|new|g pattern	Uses | as the sed delimiter (instead of /) because the image URLs contain / characters. Replaces the old image tag (any value after the colon) with the new commit SHA.
release/kubernetes-manifests.yaml	The single Kubernetes manifest file that contains all Deployment image references.

6.3 Commit and Push

```
      - name: Commit once
        run: |
          git config user.name "github-actions"
          git config user.email "actions@github.com"
          git add release/kubernetes-manifests.yaml
          git commit -m "update changed service images ${GITHUB_SHA}" || echo "no
change"
          git pull --rebase origin main
          git push origin main
```

git config user.name/email	Sets the commit author identity. Required for git commit to work.
git add release/...	Stages only the manifest file — no other files should be committed.
|| echo "no change"	If git commit exits non-zero (nothing actually changed), suppress the error and continue.
git pull --rebase origin main	Rebase the local branch on top of any remote changes made while this job was running. Prevents push rejections due to diverged history.
git push origin main	Push the manifest update. The paths-ignore: release/ trigger exclusion prevents this push from re-triggering the pipeline.

GitOps Pattern

By committing image tags to Git, the manifest file becomes the single source of truth for what is deployed. Any tool (Argo CD, Flux, or this pipeline itself) can reconcile the cluster to match the manifest.

7. Job: deploy-to-k8s — Deploy to Amazon EKS

The final job applies the updated Kubernetes manifests to the EKS cluster and verifies that each deployment rolls out successfully.

```
deploy-to-k8s:
  name: Deploy to EKS
  runs-on: ubuntu-latest
  needs: update-manifest
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
```

needs: update-manifest	Only runs after manifest is committed — ensures fresh image tags are in the manifest.
github.event_name == 'push'	Extra guard: only deploy on actual pushes, not on other event types.
github.ref == 'refs/heads/main'	Only deploy from the main branch. Feature branches should not auto-deploy to production.

7.1 OIDC Permissions for AWS

```
  permissions:
    id-token: write
    contents: read
```

id-token: write is required for GitHub's OIDC provider to issue a short-lived JWT token to the workflow. This token is exchanged for temporary AWS credentials — no long-lived access keys stored in secrets.

7.2 Environment Variables

```
  env:
    AWS_REGION: ap-south-1
    EKS_CLUSTER_NAME: my-cluster
    K8S_NAMESPACE: project
```

Job-scoped environment variables used by multiple steps. ap-south-1 is the AWS Mumbai region. The EKS cluster is named my-cluster and the app runs in the project namespace.

7.3 AWS Credentials via OIDC

```
      - name: Configure AWS credentials from GitHub OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ secrets.AWS_ROLE_TO_ASSUME }}
          aws-region: ${{ env.AWS_REGION }}
```

The configure-aws-credentials action exchanges the GitHub OIDC JWT for temporary AWS credentials by assuming the IAM role specified. The role must have a trust policy that allows GitHub Actions' OIDC provider as a federated identity.

> **OIDC vs Static Access Keys**
> Traditional approach: store AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY as GitHub secrets. OIDC approach: no long-lived credentials stored anywhere. The role assumption is time-bound (default 1 hour) and scoped to a specific repo/branch.

7.4 kubectl Setup and EKS Configuration

```
      - name: Setup kubectl
        uses: azure/setup-kubectl@v4

      - name: Generate kubeconfig for EKS
        run: |
          aws eks update-kubeconfig --region "${AWS_REGION}" --name
"${EKS_CLUSTER_NAME}"
```

azure/setup-kubectl@v4	Installs kubectl on the runner. Published by Azure but works with any Kubernetes cluster.
aws eks update-kubeconfig	Generates and writes a kubeconfig entry for the EKS cluster into ~/.kube/config using the assumed AWS credentials.
--region / --name	Specifies which EKS cluster to configure. AWS authenticates via the assumed IAM role.

7.5 Pre-flight Checks

```
      - name: Verify namespace access
        run: |
          aws sts get-caller-identity
          kubectl config current-context
          kubectl get pods -n "${K8S_NAMESPACE}"
          kubectl get services -n "${K8S_NAMESPACE}"
          kubectl auth can-i get pods -n "${K8S_NAMESPACE}"
          kubectl auth can-i create deployments -n "${K8S_NAMESPACE}"
          kubectl auth can-i update deployments -n "${K8S_NAMESPACE}"
          kubectl auth can-i create services -n "${K8S_NAMESPACE}"
```

A comprehensive pre-flight verification step that confirms the workflow has the right identity and permissions before attempting deployment.

aws sts get-caller-identity	Confirms which IAM role was assumed. Prints account ID, user ID, and ARN.
kubectl config current-context	Confirms kubectl is pointing at the right cluster.
kubectl get pods/services	Confirms cluster connectivity and namespace access.
kubectl auth can-i ...	Uses Kubernetes RBAC dry-run to confirm the service account has each required permission before attempting to use it.

7.6 Apply Manifest

```
      - name: Deploy manifest
        run: |
          kubectl apply -f release/ -n "${K8S_NAMESPACE}"
```

Applies all files in the release/ directory to the project namespace. kubectl apply is declarative — it creates resources that don't exist and updates resources that do, based on the current desired state in the YAML files.

7.7 Rollout Verification

```
      - name: Verify rollout for all deployments
```

```
        run: |
          deployments=(adservice cartservice checkoutservice currencyservice
            emailservice frontend loadgenerator paymentservice
            productcatalogservice recommendationservice shippingservice)

          for deploy in "${deployments[@]}"; do
            echo "Checking rollout for $deploy..."
            kubectl rollout status deployment/"$deploy" -n "${K8S_NAMESPACE}" --
timeout=300s
          done
```

kubectl rollout status	Blocks and watches the deployment until all pods of the new version are Ready, or the timeout expires.
--timeout=300s	Gives each deployment up to 5 minutes to finish rolling out. If a deployment is stuck (bad image, crash loop, insufficient resources), this step will fail and signal a problem.
for loop over all services	Checks every service in sequence. Note: shoppingassistantservice is missing from this list — a potential gap to address.

8. Pipeline Summary & Key Design Patterns

8.1 Security Layers

Tool	Target	What it catches
`Gitleaks`	Source code / Git history	Hardcoded secrets, tokens, API keys
`Checkov`	IaC files	Misconfigurations in Dockerfiles, K8s YAML
`Trivy FS`	Source + dependencies	CVEs in application packages
`Trivy Image`	Docker image layers	CVEs in OS packages and base images

8.2 Key Design Patterns Used

- Dynamic Matrix Strategy: Only build what changed, not all 12 services every time
- First-run Detection: Handle the edge case of a repo's first commit gracefully
- Sentinel Value (__none__): Satisfy GitHub's matrix requirement with zero real work
- fail-fast: false: Parallel service failures don't kill sibling builds
- Commit SHA as Image Tag: Every image is uniquely tied to its source code state
- GitOps Manifest Commit: Desired state lives in Git, not just in the cluster
- paths-ignore Loop Prevention: Manifest commits don't re-trigger the pipeline
- OIDC Authentication: No long-lived AWS credentials stored anywhere
- Pre-flight RBAC Checks: Verify permissions before attempting to deploy
- Rollout Status Verification: Don't declare success until pods are actually Ready

8.3 Potential Improvements

- Add --exit-code 1 to Gitleaks and Checkov to make scans blocking gates
- Add shoppingassistantservice to the rollout verification loop
- Upload Gitleaks JSON and Trivy SARIF reports to GitHub Security tab
- Add Slack/Teams notifications on failure
- Add smoke tests or integration tests after deployment
- Pin Trivy and Gitleaks to specific versions for reproducibility

DevOps Shack | @devopsshack | Deep Dive Series